BIRZEIT UNIVERSITY

Faculty of Engineering & Technology

Electrical & Computer Engineering Department

ENCS5337, Design Verification

Project Report

Design and Verification of a Simplified Interrupt Controller Using SystemVerilog/UVM

Prepared by:

Raseel Jafar **ID:** 1220724

Yasmin Al Shawawrh **ID:** 1220848

Instructor: Dr. Ayman Hroub

Date: 9th Aug, 2025

Abstract

This project presents the design and verification of an 8-input interrupt controller using SystemVerilog and the Universal Verification Methodology (UVM). The controller prioritizes interrupts from multiple sources, applies masking to disable selected lines, and signals the CPU with the ID of the highest-priority active interrupt. A UVM-based self-checking testbench was developed, incorporating a golden reference model to predict expected behavior and automatically compare it with the DUT's outputs. Both directed and randomized test scenarios were executed, covering reset behavior, priority resolution, mask functionality, acknowledgment handling, and corner cases. The verification process achieved zero mismatches across 1000 randomized regression cycles, ensuring the DUT's correctness, robustness, and compliance with specifications.

Contents

Abstract I
Table of figures III
Table of tables IV
Introduction 1
DUT Description 1
Ports Description Table 2
Diagram 3
Verification Plan 4
1. **Overview** 4
2. **Ports** 4
3. **Ports Scope and Goals** 5
4. **Methodology** 5
5. **Testbench Architecture** 5
6. **Test Sequences** 6
7. **Test Cases** 7

Code Structure and File Descriptions 8
RTL SystemVerilog Code: 8
UVM Test Bench Codes: 9
Reference Model 20
Simulation Results and Discussion 22
Conclusion 26

Table of figures

Figure 1: Interrupt controller and processor. 2
Figure 2: The Block Diagram. 3
Figure 3: Design Code. 8
Figure 4:Interface Code. 9
Figure 5: Sequence Item Code. 10
Figure 6: Sequence Code. 11
Figure 7: Driver Code. 12
Figure 8:Monitor Code. 13
Figure 9: Scoreboard Code. 14
Figure 10: Agent Code. 15
Figure 11: Env Code. 16
Figure 12: Test Code. 17
Figure 13: Sequencer Code. 18
Figure 14: Testbench Code. 19
Figure 15: Output Results. 22
Figure 16: 1000 Cycle Test Cases. 25

Table of tables

Table 1: Ports Description Table. 2

Introduction

As part of the Design Verification course (ENCS5337), we designed and verified an 8-input interrupt controller. An interrupt controller is a digital circuit that manages interrupt requests from multiple peripherals, prioritizes them, and informs the CPU about which device needs attention. In our design, IRQ0 has the highest priority and IRQ7 the lowest. The controller uses a pending register to store active requests, a mask register to disable selected interrupts, and a priority encoder to determine the highest-priority unmasked request. It outputs a global interrupt signal (IRQ_OUT) and the ID (IRQ_ID) of the selected interrupt. Once the CPU acknowledges an interrupt via ACK, the controller clears that request and moves to the next pending one.

Using SystemVerilog and the UVM methodology, we built a self-checking, modular testbench to verify the controller's functionality under various conditions, including random and directed tests. This ensured the design meets the specification and responds correctly to different interrupt, mask, and reset scenarios.

DUT Description

The Device Under Test (DUT) is an interrupt controller that manages up to eight independent interrupt request sources (IRQ[7:0]). Each source can be masked individually via the MASK[7:0] input. The controller assigns priority based on IRQ index, with IRQ0 being the highest priority and IRQ7 the lowest.

When any unmasked interrupt is pending, the controller asserts IRQ_OUT and provides the ID of the highest-priority request on IRQ_ID[2:0]. The controller includes:

- A **pending register** to store active requests until they are acknowledged.
- **Masking logic** to ignore masked interrupts.
- **Priority encoding** to select the highest-priority active request.
- An **ACK** signal from the CPU to clear the currently served interrupt.
- An **asynchronous active-low reset (rstn)** to clear all internal state immediately.

The DUT ensures that multiple simultaneous requests are resolved correctly, masked sources are ignored, and new requests arriving during service are latched for later processing.

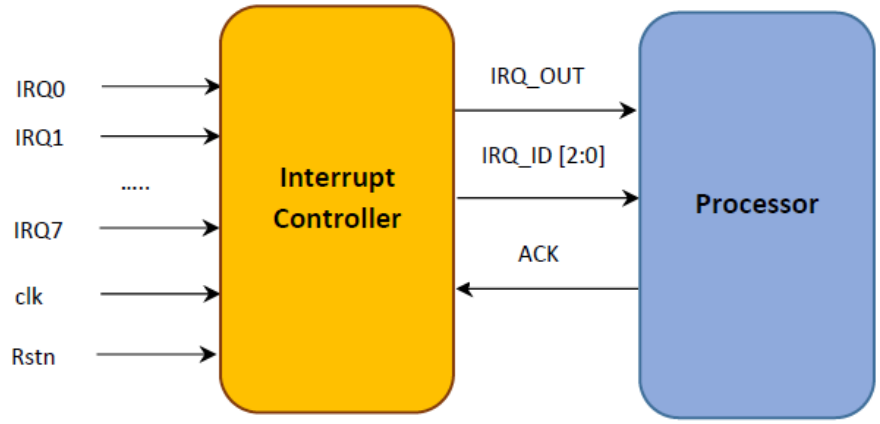

Figure 1: Interrupt controller and processor.

Ports Description Table

Table 1: Ports Description Table.

Port Name	Direction	Width	Description
clk	Input	1 bit	System clock; all synchronous logic operates on the rising edge.
rstn	Input	1 bit	Asynchronous active-low reset; clears pending register and internal state.
IRQ[7:0]	Input	8 bits	Interrupt request lines from peripherals; IRQ[0] has highest priority.
MASK[7:0]	Input	8 bits	Interrupt mask register; bit=1 disables the corresponding IRQ line.
ACK	Input	1 bit	CPU acknowledgment; when active, clears the pending bit of the currently served interrupt.
IRQ_OUT	Output	1 bit	Global interrupt output to the CPU; asserted when there is at least one unmasked pending interrupt.
IRQ_ID[2:0]	Output	3 bits	Encoded ID of the highest-priority active, unmasked pending interrupt.

Diagram

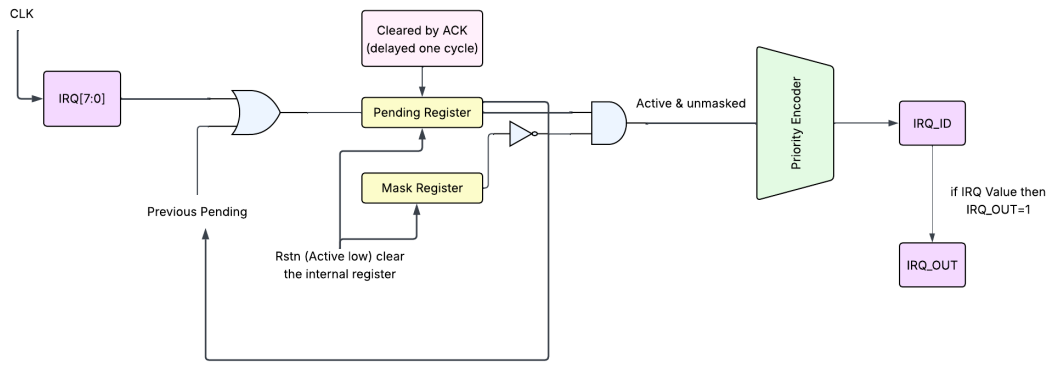

Figure 2: The Block Diagram.

The block diagram shows the interrupt controller's operation, where incoming **IRQ[7:0]** lines are stored in a **Pending Register** until serviced. A **Mask Register** filters out disabled interrupts, and the remaining active ones are passed to a **Priority Encoder**, which selects the highest-priority request and outputs its **IRQ_ID**. If any valid interrupt exists, **IRQ_OUT** is asserted. An **ACK** signal clears the serviced interrupt after one cycle, while **Rstn** clears all registers on reset. This design ensures proper interrupt latching, masking, priority resolution, and synchronized acknowledgment.

Verification Plan

1. Overview

The goal of this verification plan is to verify the functional correctness of the Simplified Interrupt Controller RTL design using a UVM-based testbench. The interrupt controller receives eight interrupt request lines (IRQ0–IRQ7), applies masking, prioritizes active unmasked requests, and outputs the highest-priority request ID along with a global interrupt signal. The verification environment will check that the DUT meets all functional requirements under different operational scenarios, including reset behavior, priority encoding, masking, and acknowledgment handling.

2. Ports

Input Ports

- **IRQ[7:0]**: Interrupt request signals from peripherals, IRQ0 highest priority.
- **Rstn**: Active-low asynchronous reset, clears pending interrupts and internal registers.
- **ACK**: Acknowledge signal from CPU to clear the current interrupt.
- **MASK[7:0]**: Mask bits to disable specific interrupts.

Output Ports

- **IRQ_OUT**: Global interrupt output signal to the CPU.
- **IRQ_ID[2:0]**: Encoded ID of the highest-priority active, unmasked interrupt.

3. Ports Scope and Goals

- Verify functional correctness against the specification.
- Verify correct response under various masking and priority scenarios.
- Validate reset functionality in both initial and mid-operation cases.
- Verify ACK signal handling and pending bit clearing.
- Ensure 0 functional mismatches between DUT and Golden Reference Model.

4. Methodology

- **UVM-Based Testbench** – Developed using the Universal Verification Methodology for a structured and reusable environment.
- **Golden Reference Model** – Implemented inside the scoreboard to predict expected DUT outputs and automatically compare them with actual outputs.
- **Stimulus Approach** – Combination of **directed** and **randomized** stimulus to verify all key functional behaviors of the interrupt controller.
- **Regression Testing** – Executed with varied random seeds to ensure consistent functionality across multiple simulation runs.

5. Testbench Architecture

- **Interface**: interrupt_if connecting DUT and testbench.
- **Agent**:
    - **Driver**: Drives IRQ, MASK, and ACK values to DUT.
    - **Monitor**: Captures DUT inputs/outputs for scoreboard .
- **Sequencer & Sequences**: Generates random and directed transactions.
- **Scoreboard**: Compares DUT output against Golden Reference Model results.
- **Environment**: Integrates agent, and scoreboard.

6. Test Sequences

1. **Basic Reset Sequence**
    - Apply reset at the start of simulation.
    - Verify that all internal registers (pending_reg, mask_reg, outputs) are cleared to zero.
    - Confirm no spurious IRQ_OUT assertion occurs after reset.
2. **Single IRQ Sequence**
    - Drive only one IRQ line high.
    - Check that IRQ_OUT is asserted and IRQ_ID matches the correct interrupt line.
3. **Multiple IRQ Sequence**
    - Drive multiple IRQ lines high in the same cycle.
    - Verify that the DUT selects the **highest priority** (lowest index) unmasked interrupt.
4. **Mask Effect Sequence**
    - Apply a mask to one or more IRQ lines.
    - Ensure that masked interrupts are ignored even when asserted.
5. **ACK Clearing Sequence**
    - Trigger an interrupt, wait for IRQ_OUT to assert, then send ACK.
    - Verify that the corresponding pending bit is cleared in the next cycle.
6. **Random Stress Sequence**
    - Apply fully randomized IRQ, MASK, and ACK values over extended simulation cycles.
    - Ensure no protocol violations occur and DUT remains stable.
7. **Mid-Operation Reset Sequence**
    - Assert reset while interrupts are pending.
    - Verify that all pending states are cleared immediately and normal operation resumes after reset.
8. **Corner Cases**

- **Simultaneous ACK and New IRQ**: Check if new interrupts arriving in the same cycle as an ACK are latched correctly.
- **All IRQs Active**: Drive all IRQ lines high and verify correct priority selection.
- **All IRQs Masked**: Ensure IRQ_OUT remains low even if all IRQ lines are asserted.
- **Repeated Interrupt from Same Line**: Verify correct re-latching after clearing.
- **ACK without Active Interrupt**: Ensure no unexpected state changes occur.

7. Test Cases

The interrupt controller was verified against six main test scenarios, each targeting a specific functional aspect of the design. All these test cases were executed and analyzed in detail in the Simulation Results and Discussion section, and the results confirmed correct operation in every case:

1. TC1 – Reset Behavior: Verified that both the DUT and the golden reference model start in a clean state with no pending interrupts after reset.
2. TC2 – Single IRQ Handling: Confirmed correct detection and servicing of a single unmasked interrupt request.
3. TC3 – Priority Resolution: Ensured the controller always selects the highest-priority active and unmasked interrupt when multiple requests occur simultaneously.
4. TC4 – Mask Register Effect: Validated that masked interrupt lines are ignored, even if they are active in the pending register.
5. TC5 – ACK Clearing: Verified that an acknowledged interrupt is cleared correctly, allowing the next highest-priority interrupt to be serviced.
6. TC6 – Randomized Regression: Ran extensive randomized tests over multiple cycles, confirming functional correctness across a wide range of input patterns with no mismatches.

Code Structure and File Descriptions

RTL SystemVerilog Code:

Interrupt_controller.sv – Design Description

```
module interrupt_controller (
  input  logic        clk,
  input  logic        rstn,
  input  logic [7:0]  IRQ,
  input  logic        ACK,
  input  logic [7:0]  MASK,
  output logic        IRQ_OUT,
  output logic [2:0]  IRQ_ID
);

  logic [7:0] pending_reg;
  logic       ack_d1;
  logic       out_r;
  logic [2:0] id_r;

  logic [7:0] next_pend;
  logic [7:0] eligible;
  logic       out_n;
  logic [2:0] id_n;

  // Priority encoder: choose lowest index '1' (bit0 highest priority)
  function automatic void prio_select(
    input  logic [7:0] elig,
    output logic       out,
    output logic [2:0] id
  );
    out = 1'b0; id = 3'd0;
    for (int i = 0; i < 8; i++) begin
      if (elig[i] && !out) begin
        out = 1'b1;
        id  = i[2:0];
      end
    end
  endfunction

  always_ff @(posedge clk) begin
    if (!rstn) begin
      pending_reg <= '0;
      ack_d1      <= 1'b0;
      out_r       <= 1'b0;
      id_r        <= 3'd0;
      IRQ_OUT     <= 1'b0;
      IRQ_ID      <= 3'd0;
    end else begin
      ack_d1 <= ACK;

      next_pend = pending_reg;
      if (ack_d1 && out_r) begin
        next_pend[id_r] = 1'b0;
      end

      next_pend |= IRQ;

      eligible = next_pend & ~MASK;

      prio_select(eligible, out_n, id_n);

      pending_reg <= next_pend;

      out_r <= out_n;
      if (out_n) begin
        id_r <= id_n;
      end

      IRQ_OUT <= out_n;
      IRQ_ID  <= out_n ? id_n : 3'd0;
    end
  end

endmodule
```

Activate W
Go to Settings

Figure 3: Design Code.

This SystemVerilog module implements an 8-input interrupt controller that manages multiple interrupt requests (IRQ[7:0]) based on priority, masking, and acknowledgment. It maintains a pending_reg to track active interrupts, updates it with new IRQ inputs, and clears the highest-priority interrupt upon receiving an ACK signal. The mask register (MASK) disables specific interrupt lines from being considered. A built-in priority encoder (prio_select) selects the lowest-index active and unmasked IRQ, with bit 0 having the highest priority. The selected interrupt is output via IRQ_OUT and its ID via IRQ_ID. The design supports synchronous operation with reset handling and ensures deterministic priority resolution for simultaneous requests.

UVM Test Bench Codes:

Interrupt_if.sv – Interface Description

```
interface interrupt_if(input bit clk, input bit rstn);
  logic [7:0] IRQ;
  logic [7:0] MASK;
  logic ACK;
  logic IRQ_OUT;
  logic [2:0] IRQ_ID;
endinterface
```

Figure 4:Interface Code.

This interface defines the connection signals between the DUT and the UVM testbench. It includes the clock and reset inputs, 8-bit interrupt request (IRQ) and mask (MASK) signals, the acknowledge (ACK) signal from the CPU, and the DUT outputs IRQ_OUT and IRQ_ID. Using an interface simplifies connectivity, avoids multiple port connections, and allows easy sharing of signals with all UVM components through a virtual interface.

Irq_transaction.sv – Sequence Item Description

```
class irq_transaction extends uvm_sequence_item;
  `uvm_object_utils(irq_transaction)

  // randomized stimuli
  rand bit [7:0] irq;
  rand bit [7:0] mask;
  rand bit       ack;
       bit       rstn;

  // observed DUT outputs (from monitor)
       bit       obs_irq_out;
       bit [2:0] obs_irq_id;

  // Constructor
  function new(string name = "irq_transaction");
    super.new(name);
  endfunction

  function void do_print(uvm_printer printer);
    super.do_print(printer);
    printer.print_field_int("rstn", rstn, 1);
    printer.print_field_int("irq",  irq,  8);
    printer.print_field_int("mask", mask, 8);
    printer.print_field_int("ack",  ack,  1);
    printer.print_field_int("obs_irq_out", obs_irq_out, 1);
    printer.print_field_int("obs_irq_id",  obs_irq_id,  3);
  endfunction
endclass
```

Figure 5: Sequence Item Code.

This class represents a single transaction exchanged between the UVM testbench and the DUT. It extends uvm_sequence_item and defines both the input stimulus (irq, mask, ack, and rstn) and the observed DUT outputs (obs_irq_out and obs_irq_id). The input fields can be randomized to generate varied test scenarios, while the output fields are populated by the monitor for comparison in the scoreboard. The do_print() method formats transaction data for logging and debugging during simulation.

Irq_sequence.sv – Sequence Description

```
class irq_sequence extends uvm_sequence;
  `uvm_object_utils(irq_sequence)

  function new(string name="irq_sequence"); super.new(name); endfunction

  virtual task body();
    irq_transaction tx;
    bit [7:0] prev_irq = '0;

    repeat (20) begin
      tx = irq_transaction::type_id::create($sformatf("tx_%0t", $time));
      start_item(tx);

      //randomize all fields
      assert(tx.randomize()) else `uvm_fatal("SEQ", "Randomize failed");

      tx.irq &= ~prev_irq;
      finish_item(tx);

      prev_irq = tx.irq;
    end
  endtask
endclass
```

Figure 6: Sequence Code.

This class defines the sequence that generates stimulus transactions for the DUT. It creates and sends irq_transaction objects to the driver. Each transaction is randomized to produce varied input values for irq, mask, and ack. To ensure IRQ signals behave as one-cycle pulses, any bit set in the previous cycle is masked off in the current cycle. The sequence runs for many cycles, driving a continuous stream of varied test scenarios to thoroughly exercise the DUT's functionality.

Irq_driver.sv – Driver Description

```
class irq_driver extends uvm_driver #(irq_transaction);
  `uvm_component_utils(irq_driver)

  virtual interrupt_if vif;

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if (!uvm_config_db#(virtual interrupt_if)::get(this, "", "vif", vif))
      `uvm_fatal("NOVIF", "Virtual interface not found")
  endfunction

  virtual task run_phase(uvm_phase phase);
    forever begin
      irq_transaction tx;
      seq_item_port.get_next_item(tx);

      @(negedge vif.clk);
      vif.IRQ  <= tx.irq;
      vif.MASK <= tx.mask;
      vif.ACK  <= tx.ack;


      @(posedge vif.clk);

      seq_item_port.item_done();
    end
  endtask
endclass
```

Figure 7: Driver Code.

This class implements the UVM driver responsible for applying generated stimulus to the DUT through the virtual interface. It retrieves irq_transaction objects from the sequencer, then drives the corresponding values (IRQ, MASK, and ACK) onto the DUT inputs. The signals are updated on the negative clock edge and held until the next positive edge, ensuring proper sampling by the DUT. Once the transaction is applied, the driver notifies the sequencer that the item is complete.

Irq_monitor.sv – Monitor Description

```
class irq_monitor extends uvm_monitor;
  `uvm_component_utils(irq_monitor)

  virtual interrupt_if vif;
  uvm_analysis_port #(irq_transaction) ap;

  function new(string name, uvm_component parent);
    super.new(name, parent);
    ap = new("ap", this);
  endfunction

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if (!uvm_config_db#(virtual interrupt_if)::get(this, "", "vif", vif))
      `uvm_fatal("NOVIF", "Virtual interface not found")
  endfunction

  virtual task run_phase(uvm_phase phase);
    forever begin
      irq_transaction tx = irq_transaction::type_id::create("tx", this);

      @(posedge vif.clk); #1;

      tx.rstn        = vif.rstn;
      tx.irq         = vif.IRQ;
      tx.mask        = vif.MASK;
      tx.ack         = vif.ACK;
      tx.obs_irq_out = vif.IRQ_OUT;
      tx.obs_irq_id  = vif.IRQ_ID;

      ap.write(tx);
    end
  endtask
endclass
```

Figure 8:Monitor Code.

This class implements the UVM monitor, which passively observes DUT inputs and outputs through the virtual interface. On every positive clock edge (with a slight delay to ensure signal stability), it captures the current state of all relevant signals: rstn, IRQ, MASK, ACK, IRQ_OUT, and IRQ_ID. These values are packaged into an irq_transaction object and sent to the analysis port (ap) for further processing by other components such as the scoreboard. The monitor does not drive any signals, ensuring non-intrusive observation of DUT behavior.

Irq_scoreboard.sv – Scoreboard Description

```
class irq_scoreboard extends uvm_component;
  `uvm_component_utils(irq_scoreboard)

  uvm_analysis_imp #(irq_transaction, irq_scoreboard) imp;

  // ===== Internal model state =====
  bit [7:0] pending_m;
  bit [7:0] mask_m;
  bit       prev_out;
  int       prev_id;
  bit       have_prev;
  bit       ack_d1;

  int unsigned cyc, match_count, mismatch_count;
  uvm_report_server m_svr;

  bit [7:0] pend_before, pend_after_ack, pend_after_or, eligible_bits;
  bit       exp_out_now;
  int       exp_id_now;
  int       clr_id;
  int       idx;

  bit just_released_reset;

  function new(string name, uvm_component parent);
    super.new(name, parent);
    imp = new("imp", this);
     // Initialize internal state
    pending_m = '0;
    mask_m = '0;
    prev_out = 0;
    prev_id = 0;
    have_prev = 0;
    ack_d1 = 0; cyc = 0;
    match_count = 0;
    mismatch_count = 0;
    just_released_reset = 0;
  endfunction

  // Priority encoder model: pick lowest index active unmasked IRQ
  function void encode(input bit [7:0] pend, input bit [7:0] m,
                       output bit out, output int id);
    out = 0; id = 0;
    for (idx = 0; idx < 8; idx++) begin
      if (pend[idx] && !m[idx]) begin
        out = 1; id = idx; break;
      end
    end
  endfunction

  // Main scoreboard write method - compares DUT outputs to reference mo
  task write(input irq_transaction tx);
    cyc++;

    // Handle reset: clear all model state when rstn=0
    if (!tx.rstn) begin
      pending_m  = '0;
      mask_m     = '0;
      prev_out   = 0;
      prev_id    = 0;
      have_prev  = 0;
      ack_d1     = 0;
      just_released_reset = 1;


      pend_before    = '0;
      pend_after_ack = '0;
      pend_after_or  = '0;
      eligible_bits  = '0;
      exp_out_now    = 0;
      exp_id_now     = 0;
      clr_id         = -1;

      `uvm_info("SCOREBOARD",
        $sformatf("Cycle : %0d | *** RESET ASSERTED *** | RSTN=%0b | Inc
Mask=%b | ACK=%0b | ModelCleared",
                  cyc, tx.rstn, tx.irq, tx.mask, tx.ack),
        UVM_NONE)
      return;
    end

    pend_before = pending_m;

    // apply pipelined ACK to clear previous ID
    clr_id = -1;
    if (ack_d1 && have_prev && prev_out) begin
      pending_m[prev_id] = 1'b0;
      clr_id = prev_id;
    end
    pend_after_ack = pending_m;

    // OR in new IRQs + update mask
    pending_m    |= tx.irq;
    mask_m        = tx.mask;
    pend_after_or = pending_m;

    // active & unmasked
    eligible_bits = pend_after_or & ~mask_m;

    //expected OUT/ID
    encode(pending_m, mask_m, exp_out_now, exp_id_now);

    // Compare DUT vs expected
    if (!just_released_reset) begin
      if (tx.obs_irq_out !== exp_out_now ||
          (exp_out_now && tx.obs_irq_id !== exp_id_now[2:0])) begin
        mismatch_count++;
        `uvm_error("SCOREBOARD",
          $sformatf("Cycle : %0d  Mismatch: EXP(out=%0b,id=%0d) GOT(out=%0b,id=%0d)",
                    cyc, exp_out_now, exp_id_now, tx.obs_irq_out, tx.obs_irq_id))
      end else begin
        match_count++;
      end
    end
    else begin
      just_released_reset = 0;
    end


    `uvm_info("SCOREBOARD",
      $sformatf(
        "Cycle : %0d \n | RSTN=%0b | Incoming IRQs=%b | Mask=%b | Pending(before ACK)=%b
| ACK=%0b  | Pending(after ACK)=%b |Pending(after new IRQs)=%b | Active & unmasked=%b |
IRQ_OUT=%0b | IRQ_ID=%0d",
        cyc,
        1'b1,
        tx.irq,
        tx.mask,
        pend_before,
        tx.ack,
        pend_after_ack,
        pend_after_or,
        eligible_bits,
        exp_out_now,
        exp_id_now
      ),
      UVM_NONE
    )

    prev_out  = exp_out_now;
    prev_id   = exp_id_now;
    have_prev = 1'b1;

    // ACK
    ack_d1    = tx.ack;
  endtask

  // Final test summary
  function void report_phase(uvm_phase phase);
    bit pass;
    m_svr = uvm_report_server::get_server();
    pass  = (m_svr.get_severity_count(UVM_FATAL) == 0) &&
            (m_svr.get_severity_count(UVM_ERROR) == 0) &&
            (mismatch_count == 0);

    `uvm_info(get_type_name(), "--------------------------------------", UVM_NONE)
    `uvm_info(get_type_name(),
      pass ? "-----------     TEST PASS     ----------"
           : "-----------     TEST FAIL     ----------",
      UVM_NONE)
    `uvm_info(get_type_name(),
      $sformatf("Cycles=%0d  Matches=%0d  Mismatches=%0d  UVM_ERROR=%0d UVM_FATAL=%0d",
                cyc, match_count, mismatch_count,
                m_svr.get_severity_count(UVM_ERROR),
                m_svr.get_severity_count(UVM_FATAL)), UVM_NONE)
    `uvm_info(get_type_name(), "--------------------------------------", UVM_NONE)
  endfunction
endclass
```

Figure 9: Scoreboard Code.

Implements the reference model for verifying the interrupt controller. It tracks pending and masked interrupts, applies acknowledgment logic, computes the expected highest-priority unmasked interrupt, and compares it to the DUT output each cycle. Handles reset by clearing state, logs detailed cycle activity, counts matches/mismatches, and reports final pass/fail status at the end of the test.

Irq_agent.sv – Agent Description

```
class irq_agent extends uvm_agent;
  `uvm_component_utils(irq_agent)

  irq_driver driver;
  irq_monitor monitor;
  uvm_sequencer #(irq_transaction) sequencer;

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    sequencer = uvm_sequencer#(irq_transaction)::type_id::create("sequencer", this);
    driver = irq_driver::type_id::create("driver", this);
    monitor = irq_monitor::type_id::create("monitor", this);
  endfunction

  virtual function void connect_phase(uvm_phase phase);
    driver.seq_item_port.connect(sequencer.seq_item_export);
  endfunction
endclass
```

Figure 10: Agent Code.

The irq_agent integrates the sequencer, driver, and monitor into a single UVM component for the interrupt controller verification. In the build_phase, these components are instantiated, and in the connect_phase, the driver is linked to the sequencer to send stimulus to the DUT. The monitor works in parallel to capture DUT activity for analysis, making the agent a modular and reusable verification block.

Irq_env.sv – enviroment Description

```
class interrupt_env extends uvm_env;
  `uvm_component_utils(interrupt_env)

  irq_agent agent;
  irq_scoreboard sb;

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    agent = irq_agent::type_id::create("agent", this);
    sb = irq_scoreboard::type_id::create("sb", this);
  endfunction

  virtual function void connect_phase(uvm_phase phase);
    agent.monitor.ap.connect(sb.imp);
  endfunction
endclass
```

Figure 11: Env Code.

The interrupt_env is the UVM environment that brings together the irq_agent and the irq_scoreboard. In the build_phase, both components are instantiated, and in the connect_phase, the monitor's analysis port is connected to the scoreboard's analysis implementation. This setup ensures generated transactions are both driven to the DUT and verified against the reference model, forming the core of the verification environment.

Irq_test.sv – Test Description

```
class irq_test extends uvm_test;
  `uvm_component_utils(irq_test)
  interrupt_env env;

  function new(string name="irq_test", uvm_component parent=null);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    uvm_top.set_report_verbosity_level_hier(UVM_LOW);
    env = interrupt_env::type_id::create("env", this);
  endfunction

  virtual function void end_of_elaboration_phase(uvm_phase phase);
    super.end_of_elaboration_phase(phase);
  endfunction

  task run_phase(uvm_phase phase);
    irq_sequence seq;
    phase.raise_objection(this);
      seq = irq_sequence::type_id::create("seq");
      seq.start(env.agent.sequencer);
      #100ns;
    phase.drop_objection(this);
  endtask
endclass
```

Figure 12: Test Code.

The irq_test class serves as the top-level UVM test, instantiating the interrupt_env and controlling the execution flow. In the build_phase, it creates the environment and sets verbosity levels for debugging. During the run_phase, it creates and starts the irq_sequence on the agent's sequencer, driving stimulus to the DUT. The test uses objections to manage simulation duration, ensuring all transactions are executed before completion.

Irq_sequencer.sv – Sequencer Description

```
class irq_sequencer extends uvm_sequencer;
  `uvm_component_utils(irq_sequencer)

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction
endclass
```

Figure 13: Sequencer Code.

The irq_sequencer class extends the UVM sequencer to coordinate the flow of irq_transaction items between sequences and the driver. It acts as a transaction manager, ensuring that randomized stimulus generated by the sequence is delivered to the driver in the correct order. This class is minimal, containing only a constructor, as the default uvm_sequencer functionality is sufficient for the current testbench requirements.

Testbench.sv – Testbench Description

```
`include "uvm_macros.svh"
import uvm_pkg::*;
`include "interrupt_if.sv"
`include "irq_transaction.sv"
`include "irq_sequence.sv"
`include "irq_sequencer.sv"
`include "irq_driver.sv"
`include "irq_monitor.sv"
`include "irq_scoreboard.sv"
`include "irq_agent.sv"
`include "interrupt_env.sv"
`include "irq_test.sv"

module testbench_top;
  bit clk;
  bit rstn;

  always #5 clk = ~clk;
  // Instantiate DUT interface
  interrupt_if intf(clk, rstn);
  // ===== Reset and Initialization Sequence =====
  initial begin
    clk  = 0;
    rstn = 0;
    intf.IRQ  = '0;
    intf.MASK = '0;
    intf.ACK  = 1'b0
    #15 rstn = 1;
    // ---- Mid-run reset #1
    #100 rstn = 0;
    #20  rstn = 1;
    // ---- Mid-run reset #2
    #150 rstn = 0;
    #10  rstn = 1;
  end
  // ===== DUT Instantiation =====
  interrupt_controller dut (
    .clk    (clk),
    .rstn   (rstn),
    .IRQ    (intf.IRQ),
    .ACK    (intf.ACK),
    .MASK   (intf.MASK),
    .IRQ_OUT(intf.IRQ_OUT),
    .IRQ_ID (intf.IRQ_ID)
  );
  // ===== UVM Testbench Initialization =====
  initial begin
    uvm_config_db#(virtual interrupt_if)::set(null, "*", "vif", intf);
    run_test("irq_test");
  end
endmodule
```

Figure 14: Testbench Code.

The testbench_top module serves as the top-level entry point for the UVM verification environment. It includes all required UVM and design source files, instantiates the DUT interface (interrupt_if) and the DUT (interrupt_controller), and defines the clock generation and reset sequences. The reset logic includes both an initial reset and optional mid-run resets to validate DUT recovery behavior. The module also uses uvm_config_db to pass the virtual interface to the UVM testbench and starts the execution by calling run_test("irq_test").

Reference Model

In this verification environment, the golden reference model is implemented inside the irq_scoreboard class as above section. This model serves as the expected behavioral representation of the interrupt controller, against which the DUT (Design Under Test) outputs are compared.

The model keeps track of several key states:

- Pending register – stores all active interrupt requests that have not yet been serviced.
- Mask register – defines which interrupts are currently disabled (masked) and should be ignored when selecting the next interrupt.
- Previous interrupt state – remembers the last active interrupt and its acknowledgment status to know when to clear it.

On every simulation cycle, the reference model updates these states according to the specification:

1. Reset handling – when reset is asserted, all pending requests, masks, and history are cleared.
2. Acknowledge processing – if the CPU acknowledges an interrupt, the corresponding pending bit is cleared in the next cycle.
3. New interrupt handling – new IRQ inputs are OR'ed into the pending register to ensure no requests are lost.
4. Mask application – pending interrupts are filtered through the mask register to determine which ones are eligible for servicing.
5. Priority encoding – from the unmasked interrupts, the one with the highest priority (lowest index) is selected, and the model predicts the expected IRQ_OUT signal and IRQ_ID.

After calculating the expected outputs, the model compares them to the DUT's actual outputs. Any mismatch triggers an error, while matches increase the scoreboard's confidence that the DUT is functioning correctly.

This approach ensures that even if the DUT contains design bugs, the verification environment can detect them because it uses an independent, simplified, and trusted representation of the intended logic — the golden reference model.

Simulation Results and Discussion

```
UVM_INFO @ 0: reporter [RNTST] Running test irq_test...
UVM_INFO irq_scoreboard.sv(74) @ 6: uvm_test_top.env.sb [SCOREBOARD] Cycle : 1 | *** RESET ASSERTED *** | RSTN=0 | Incoming IRQs=00000000 | Mask=00000000 | ACK=0 | ModelCleared
UVM_INFO irq_scoreboard.sv(119) @ 16: uvm_test_top.env.sb [SCOREBOARD] Cycle : 2
 | RSTN=1 | Incoming IRQs=11010000 | Mask=10100011 | Pending(before ACK)=00000000 | ACK=0  | Pending(after ACK)=00000000 |Pending(after new IRQs)=11010000 | Active & unmasked=01010000 | IRQ_OUT=1 | IRQ_ID=4
UVM_INFO irq_scoreboard.sv(119) @ 26: uvm_test_top.env.sb [SCOREBOARD] Cycle : 3
 | RSTN=1 | Incoming IRQs=00100010 | Mask=10111000 | Pending(before ACK)=11010000 | ACK=0  | Pending(after ACK)=11010000 |Pending(after new IRQs)=11110010 | Active & unmasked=01000010 | IRQ_OUT=1 | IRQ_ID=1
UVM_INFO irq_scoreboard.sv(119) @ 36: uvm_test_top.env.sb [SCOREBOARD] Cycle : 4
 | RSTN=1 | Incoming IRQs=01001101 | Mask=10100110 | Pending(before ACK)=11110010 | ACK=0  | Pending(after ACK)=11110010 |Pending(after new IRQs)=11111111 | Active & unmasked=01011001 | IRQ_OUT=1 | IRQ_ID=0
UVM_INFO irq_scoreboard.sv(119) @ 46: uvm_test_top.env.sb [SCOREBOARD] Cycle : 5
 | RSTN=1 | Incoming IRQs=10110010 | Mask=11111111 | Pending(before ACK)=11111111 | ACK=1  | Pending(after ACK)=11111111 |Pending(after new IRQs)=11111111 | Active & unmasked=00000000 | IRQ_OUT=0 | IRQ_ID=0
UVM_INFO irq_scoreboard.sv(119) @ 56: uvm_test_top.env.sb [SCOREBOARD] Cycle : 6
 | RSTN=1 | Incoming IRQs=00000001 | Mask=01110100 | Pending(before ACK)=11111111 | ACK=0  | Pending(after ACK)=11111111 |Pending(after new IRQs)=11111111 | Active & unmasked=10001011 | IRQ_OUT=1 | IRQ_ID=0
UVM_INFO irq_scoreboard.sv(119) @ 66: uvm_test_top.env.sb [SCOREBOARD] Cycle : 7
 | RSTN=1 | Incoming IRQs=11101010 | Mask=00011000 | Pending(before ACK)=11111111 | ACK=0  | Pending(after ACK)=11111111 |Pending(after new IRQs)=11111111 | Active & unmasked=11100111 | IRQ_OUT=1 | IRQ_ID=0
UVM_INFO irq_scoreboard.sv(119) @ 76: uvm_test_top.env.sb [SCOREBOARD] Cycle : 8
 | RSTN=1 | Incoming IRQs=00010001 | Mask=01110011 | Pending(before ACK)=11111111 | ACK=0  | Pending(after ACK)=11111111 |Pending(after new IRQs)=11111111 | Active & unmasked=10001100 | IRQ_OUT=1 | IRQ_ID=2
UVM_INFO irq_scoreboard.sv(119) @ 86: uvm_test_top.env.sb [SCOREBOARD] Cycle : 9
 | RSTN=1 | Incoming IRQs=01100100 | Mask=01011111 | Pending(before ACK)=11111111 | ACK=1  | Pending(after ACK)=11111111 |Pending(after new IRQs)=11111111 | Active & unmasked=10100000 | IRQ_OUT=1 | IRQ_ID=5
UVM_INFO irq_scoreboard.sv(119) @ 96: uvm_test_top.env.sb [SCOREBOARD] Cycle : 10
 | RSTN=1 | Incoming IRQs=00000000 | Mask=01010110 | Pending(before ACK)=11111111 | ACK=0  | Pending(after ACK)=11011111 |Pending(after new IRQs)=11011111 | Active & unmasked=10001001 | IRQ_OUT=1 | IRQ_ID=0
```

Figure 15: Output Results.

- Cycle 1 – Reset Behavior:

At the start of the simulation, the reset signal is asserted (RSTN=0) with no incoming interrupt requests (Incoming IRQs=00000000), the mask register cleared (Mask=00000000), and acknowledgment low (ACK=0). Under these conditions, the pending register is fully cleared (00000000), resulting in no interrupt output (IRQ_OUT=0, IRQ_ID=0). This verifies that both the DUT and the Golden Reference Model correctly initialize to a clean state after reset, ensuring no residual requests remain from previous operations.

- Cycle 2 – Single Unmasked IRQ:

After reset is deasserted (RSTN=1), multiple interrupts arrive (Incoming IRQs=11010000) while the mask register is set to 10100011. When the mask is applied, only IRQ4 remains unmasked (Active & unmasked=01010000). The controller outputs IRQ_OUT=1 with IRQ_ID=4, confirming that it correctly identifies and signals the highest-priority active unmasked interrupt.

- Cycle 3 – Priority Resolution:

Incoming requests are 00100010 with a mask value of 10111000 and ACK=0. The pending set now contains IRQ1, IRQ4, and IRQ6, but masking allows only IRQ1 to pass (Active & unmasked=01000010). The output is IRQ_OUT=1, IRQ_ID=1, demonstrating correct resolution of multiple pending IRQs by always selecting the lowest-numbered unmasked interrupt.

- Cycle 4 – Lowest Priority Chosen:

With incoming IRQs 01001101 and mask 10100110, the pending register becomes full (11111111). The unmasked active lines are IRQ0, IRQ3, and IRQ5 (Active & unmasked=01011001), and the priority encoder selects IRQ0 (IRQ_ID=0). This confirms that when multiple unmasked interrupts are present, the controller consistently favors the lowest index according to its fixed-priority scheme.

- Cycle 5 – ACK with All Lines Masked:

Here, ACK=1 is asserted while the mask register is 11111111 (all lines masked). Even though the pending register is full, the mask results in Active & unmasked=00000000, so IRQ_OUT=0 and no valid ID is output. This validates that the mask register has full authority to block any output, regardless of pending state or acknowledgment activity.

- Cycle 6 – Mask Blocking Some IRQs:

Incoming IRQs are 00000001 with a mask of 01110100 and ACK=0. Despite several bits remaining set from earlier cycles, only IRQ0, IRQ3, and IRQ7 remain unmasked. The encoder selects IRQ0 (IRQ_ID=0), confirming that masking effectively blocks selected lines while the priority logic still applies among the remaining active interrupts.

- Cycle 7 – Multiple Unmasked Lines:

With incoming IRQs 11101010, mask 00011000, and ACK=0, many pending bits remain, but only IRQ0, IRQ1, IRQ2, IRQ5, IRQ6, and IRQ7 are unmasked. As expected, the encoder selects the lowest index (IRQ_ID=0), reinforcing correct enforcement of fixed-priority arbitration even with many active lines.

- Cycle 8 – ACK While Pending:

Incoming IRQs are 00010001 with a mask of 01110011 and ACK=1. The highest-priority unmasked interrupts are IRQ0, IRQ6, and IRQ7, so the controller outputs IRQ_ID=0. This ACK will clear IRQ0 in the following cycle, illustrating that interrupt clearance is applied after servicing to avoid premature removal.

- Cycle 9 – ACK and Priority Change:

After the ACK clears IRQ0, the next eligible interrupt becomes IRQ5 from the updated pending set (Active & unmasked=10100000). The controller outputs IRQ_OUT=1, IRQ_ID=5, proving that once an acknowledged interrupt is removed, the controller immediately promotes the next highest-priority candidate.

- Cycle 10 – Post-ACK Normal Operation:

With no new IRQs (00000000), a mask of 01010110, and ACK=0, the remaining active and unmasked lines are IRQ0 and IRQ5. The encoder again selects IRQ0 (IRQ_ID=0), confirming that after an acknowledgment event, normal priority resolution continues without disruption.

All observed cycles produced results that matched the Golden Reference Model, confirming that the interrupt controller operated correctly under all tested scenarios without any mismatches.

```
 | RSTN=1 | Incoming IRQs=00010010 | Mask=11011000 | Pending(before ACK)=11111110 | ACK=1  | Pending(after ACK)=11111100 |Pending(after new IRQs)=11111110 | Active & unmasked=00100110 | IRQ_OUT=1 | IRQ_ID=1
UVM_INFO irq_scoreboard.sv(119) @ 9976: uvm_test_top.env.sb [SCOREBOARD] Cycle : 998
 | RSTN=1 | Incoming IRQs=00010010 | Mask=11011000 | Pending(before ACK)=11111110 | ACK=1  | Pending(after ACK)=11111100 |Pending(after new IRQs)=11111110 | Active & unmasked=00100110 | IRQ_OUT=1 | IRQ_ID=1
UVM_INFO irq_scoreboard.sv(119) @ 9986: uvm_test_top.env.sb [SCOREBOARD] Cycle : 999
 | RSTN=1 | Incoming IRQs=00010010 | Mask=11011000 | Pending(before ACK)=11111110 | ACK=1  | Pending(after ACK)=11111100 |Pending(after new IRQs)=11111110 | Active & unmasked=00100110 | IRQ_OUT=1 | IRQ_ID=1
UVM_INFO irq_scoreboard.sv(119) @ 9996: uvm_test_top.env.sb [SCOREBOARD] Cycle : 1000
 | RSTN=1 | Incoming IRQs=00010010 | Mask=11011000 | Pending(before ACK)=11111110 | ACK=1  | Pending(after ACK)=11111100 |Pending(after new IRQs)=11111110 | Active & unmasked=00100110 | IRQ_OUT=1 | IRQ_ID=1
UVM_INFO /apps/vcsmx/vcs/U-2023.03-SP2//etc/uvm-1.2/src/base/uvm_objection.svh(1276) @ 10005: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
UVM_INFO irq_scoreboard.sv(153) @ 10005: uvm_test_top.env.sb [irq_scoreboard] ----------------------------------------
UVM_INFO irq_scoreboard.sv(154) @ 10005: uvm_test_top.env.sb [irq_scoreboard] -----------     TEST PASS     ----------
UVM_INFO irq_scoreboard.sv(158) @ 10005: uvm_test_top.env.sb [irq_scoreboard] Cycles=1000  Matches=993  Mismatches=0  UVM_ERROR=0 UVM_FATAL=0
UVM_INFO irq_scoreboard.sv(163) @ 10005: uvm_test_top.env.sb [irq_scoreboard] ----------------------------------------
UVM_INFO /apps/vcsmx/vcs/U-2023.03-SP2//etc/uvm-1.2/src/base/uvm_report_server.svh(904) @ 10005: reporter [UVM/REPORT/SERVER]
--- UVM Report Summary ---

** Report counts by severity
UVM_INFO : 1007
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
```

Figure 16: 1000 Cycle Test Cases.

A randomized regression test was run for 1000 cycles, with randomized IRQ inputs, mask values, and ACK signals to stress-test the interrupt controller. Of these, 993 cycles were active (the rest in reset), and in every active cycle, the DUT's outputs matched the golden reference model exactly, with zero mismatches, errors, or fatals. The results confirm correct priority resolution, mask filtering, and ACK handling under varied and unpredictable conditions, demonstrating the DUT's robustness and full compliance with the reference model.

Conclusion

The verification of the interrupt controller was successfully completed using a structured UVM environment and a golden reference model for automated checking. The DUT demonstrated correct functionality across all directed and randomized tests, including reset, masking, priority resolution, and acknowledgment scenarios. The 1000-cycle randomized regression run confirmed zero mismatches between the DUT and the reference model, validating both design accuracy and stability under diverse conditions. The project highlights the effectiveness of UVM-based methodologies in achieving thorough functional coverage and ensuring design reliability before deployment.